Light

GCSE Computer Science – Algorithms and Trace Tables (Set 1)

Over **1,100,000+ marks** of exam-aligned content across **67 qualifications**. 4,200+ auto-marked assessments & long-answer practice. No login required.

GCSE • AQA 8525 • Grades 3—9 | Topic test | 30 min | 55 marks | Answered: 0 / 30

00:00 Start Pause Reset

← Back to Algorithms Trace Tables

Multiple-choice topic test for **GCSE Computer Science (AQA 8525)**. You may skip questions. When you submit, you'll see your score, a grade estimate, and worked explanations. **Tip:** use the stopwatch above to time yourself.

Q1 (1 mark)

What is the primary purpose of a trace table when analysing an algorithm?

- ○ **A.** To track how variable values change as the algorithm executes step by step
- ○ **B.** To make the algorithm run faster
- ○ **C.** To convert the algorithm into machine code
- ○ **D.** To delete unnecessary lines of code

Q2 (1 mark)

Which of the following best describes an algorithm?

- ○ **A.** A type of computer hardware
- ○ **B.** A programming language
- ○ **C.** The physical memory inside a computer
- ○ **D.** A sequence of steps that can be followed to complete a task

Q3 (1 mark)

In a FOR loop written as `FOR i ← 1 TO 5`, how many times will the loop body execute?

- ○ **A.** 4 times
- ○ **B.** 5 times
- ○ **C.** 6 times
- ○ **D.** 0 times

Q4 (1 mark)

What is the result of the expression `17 MOD 5`?

- ○ **A.** 3
- ○ **B.** 5
- ○ **C.** 2
- ○ **D.** 17

Q5 (1 mark)

What is the result of the expression `23 DIV 4`?

- ○ **A.** 5.75
- ○ **B.** 5
- ○ **C.** 3
- ○ **D.** 6

Q6 (1 mark)

In a WHILE loop, when is the condition checked?

- ○ **A.** At the start of each iteration, before the loop body executes
- ○ **B.** Only once at the very end
- ○ **C.** Randomly during execution
- ○ **D.** Never - WHILE loops run forever

Q7 (1 mark)

In an IF-THEN-ELSE structure, when does the ELSE branch execute?

- ○ **A.** When the IF condition is true
- ○ **B.** Always, regardless of the condition
- ○ **C.** When the IF condition is false

- ○ **D.** Only on Mondays

Q8 (1 mark)

Consider this code:

```
x ← 5
x ← x + 3
```

What is the final value of x?

- ○ **A.** 5
- ○ **B.** 8
- ○ **C.** 3
- ○ **D.** 53

Q9 (1 mark)

In the context of algorithms, what is meant by 'processing'?

- ○ **A.** Receiving data from the user
- ○ **B.** Displaying results on screen
- ○ **C.** The operations performed on data to transform inputs into outputs
- ○ **D.** Saving data to a file

Q10 (1 mark)

An array `numbers` contains the values [10, 20, 30, 40, 50]. What value does `numbers[2]` return if arrays are indexed starting from 0?

- ○ **A.** 20
- ○ **B.** 10
- ○ **C.** 40
- ○ **D.** 30

Q11 (1 mark)

What does 'decomposition' mean in the context of computational thinking?

- ○ **A.** Breaking a problem into smaller, more manageable sub-problems
- ○ **B.** Making a program run faster
- ○ **C.** Removing all variables from a program
- ○ **D.** Converting code to binary

Q12 (1 mark)

What is abstraction in computational thinking?

- ○ **A.** Adding more detail to make problems more complex
- ○ **B.** Writing code in abstract art style

- ○ **C.** Making algorithms longer
- ○ **D.** The process of removing unnecessary detail from a problem to focus on what is important

Q13 (2 marks)

Trace the following algorithm:

```
total ← 0
FOR i ← 1 TO 4
    total ← total + i
ENDFOR
```

What is the final value of total?

- ○ **A.** 4
- ○ **B.** 16
- ○ **C.** 0
- ○ **D.** 10

Q14 (2 marks)

Trace the following algorithm:

```
x ← 10
WHILE x > 3
    x ← x - 2
ENDWHILE
```

What is the final value of x?

- ○ **A.** 2

- B. 3
- C. 4
- D. 0

Q15 (2 marks)

Trace the following algorithm with input value 15:

```
INPUT num
IF num > 20 THEN
    result ← "large"
ELSE
    IF num > 10 THEN
        result ← "medium"
    ELSE
        result ← "small"
    ENDIF
ENDIF
```

What is the value of result?

- A. large
- B. small
- C. 15
- D. medium

Q16 (2 marks)

What does this algorithm determine about its input?

```
INPUT n
IF n MOD 2 = 0 THEN
```

```
    OUTPUT "A"
ELSE
    OUTPUT "B"
ENDIF
```

- A. Whether n is positive or negative
- B. Whether n is even or odd
- C. Whether n is greater than 2
- D. Whether n is a prime number

Q17 (2 marks)

Trace this algorithm with the array values [5, 8, 3, 9, 2]:

```
count ← 0
FOR i ← 0 TO 4
    IF values[i] > 4 THEN
        count ← count + 1
    ENDIF
ENDFOR
```

What is the final value of count?

- A. 2
- B. 5
- C. 4
- D. 3

Q18 (2 marks)

What does this algorithm calculate?

```
result ← 1
FOR i ← 1 TO n
    result ← result * i
ENDFOR
```

- ○ **A.** The factorial of n (n!)
- ○ **B.** The sum of numbers from 1 to n
- ○ **C.** The value of n squared
- ○ **D.** The average of numbers from 1 to n

Q19 (2 marks)

Consider this algorithm with input "Hello":

```
INPUT word
length ← LEN(word)
result ← length * 2
```

What is the value of result?

- ○ **A.** 5
- ○ **B.** Hello
- ○ **C.** 10
- ○ **D.** HelloHello

Q20 (2 marks)

Trace this algorithm:

```
x ← 1
REPEAT
    x ← x * 2
UNTIL x > 10
```

What is the final value of x?

- A. 8
- B. 10
- C. 12
- D. 16

Q21 (2 marks)

In a flowchart, what shape is used to represent a decision (such as an IF statement)?

- A. Rectangle
- B. Oval
- C. Diamond
- D. Parallelogram

Q22 (2 marks)

Trace this algorithm that swaps two values:

```
a ← 5
b ← 8
temp ← a
```

```
a ← b
b ← temp
```

What are the final values of a and b?

- ○ **A.** a = 5, b = 8
- ○ **B.** a = 8, b = 8
- ○ **C.** a = 5, b = 5
- ○ **D.** a = 8, b = 5

Q23 (2 marks)

What does this algorithm find?

```
max ← values[0]
FOR i ← 1 TO LEN(values) - 1
    IF values[i] > max THEN
        max ← values[i]
    ENDIF
ENDFOR
```

- ○ **A.** The minimum value in the array
- ○ **B.** The maximum value in the array
- ○ **C.** The sum of all values in the array
- ○ **D.** The average of all values in the array

Q24 (3 marks)

Trace this nested loop algorithm completely:

```
total ← 0
FOR i ← 1 TO 2
    FOR j ← 1 TO 3
        total ← total + 1
    ENDFOR
ENDFOR
```

What is the final value of total?

- A. 3
- B. 5
- C. 2
- D. 6

Q25 (3 marks)

Trace this algorithm with inputs age = 25 and member = TRUE:

```
INPUT age, member
IF age >= 18 AND member = TRUE THEN
    price ← 10
ELSE
    IF age >= 18 THEN
        price ← 15
    ELSE
        price ← 5
    ENDIF
ENDIF
```

What is the value of price?

- A. 10
- B. 5

- **C.** 15
- **D.** 25

Q26 (3 marks)

Trace this algorithm that modifies an array:

```
data ← [2, 4, 6, 8]
FOR i ← 0 TO 3
    data[i] ← data[i] + i
ENDFOR
```

What is the final state of the array?

- **A.** [2, 4, 6, 8]
- **B.** [3, 5, 7, 9]
- **C.** [0, 1, 2, 3]
- **D.** [2, 5, 8, 11]

Q27 (3 marks)

Analyse what this algorithm outputs for input 347:

```
INPUT num
WHILE num > 0
    digit ← num MOD 10
    OUTPUT digit
    num ← num DIV 10
ENDWHILE
```

- **A.** 347

- ○ **B.** 3, 4, 7 (digits in original order)
- ○ **C.** 7, 4, 3 (digits in reverse order)
- ○ **D.** 14 (sum of digits)

Q28 (3 marks)

Trace this algorithm that processes a string:

```
word ← "CODE"
result ← ""
FOR i ← LEN(word) - 1 DOWNTO 0
    result ← result + word[i]
ENDFOR
```

What is the final value of result?

- ○ **A.** CODE
- ○ **B.** CCCC
- ○ **C.** EDOC
- ○ **D.** E

Q29 (3 marks)

Analyse the purpose of this algorithm:

```
found ← FALSE
position ← -1
i ← 0
WHILE i < LEN(data) AND found = FALSE
    IF data[i] = target THEN
        found ← TRUE
        position ← i
```

```
    ENDIF
    i ← i + 1
ENDWHILE
```

- ○ **A.** It sorts the array data in ascending order
- ○ **B.** It performs a linear search, finding the position of target in data, or -1 if not found
- ○ **C.** It counts how many times target appears in data
- ○ **D.** It reverses the array data

Q30 (3 marks)

Consider this 2D array and algorithm:

```
grid ← [[1, 2, 3],
        [4, 5, 6],
        [7, 8, 9]]
total ← 0
FOR row ← 0 TO 2
    total ← total + grid[row][row]
ENDFOR
```

What is the final value of total?

- ○ **A.** 45
- ○ **B.** 12
- ○ **C.** 6
- ○ **D.** 15

Williams Physics Education — over **1,100,000+ marks** of exam-aligned content and **4,200+ auto-marked assessments** across 67 qualifications at GCSE, A-Level, AP & Professional Qualifications. Free to use, no login required.

About & Contact · Privacy · Terms · Accessibility · Copyright · DMCA · YouTube

WPE is free for everyone. If our resources help you or your students, you can support the project here.

© 2026 Williams Physics Education. All rights reserved.